

mochi — 단계적 구현 순서 및 코드 작성 규약

로터리 압축기 시뮬레이션 프로젝트 · PLAN.md / PHYSICS.md / ARCHITECTURE.md / AGENTS.md / CONTRIBUTING.md / README.md 요약 · 2026-07-16

문서 간 우선순위 (충돌 시 임의로 하나를 고르지 말고 불일치를 보고·해결):

① 테스트 & 검토된 기준 케이스 → ② PHYSICS.md (수학·수치 모델 계약) → ③ ARCHITECTURE.md (모듈 경계) → ④ PLAN.md (의도된 작업 기록)

제1부 — 단계적 구현 순서 (PLAN.md)

Phase 0 — 저장소 기반 ☒ 완료

- 설치 가능한 Python 패키지 셀, 포맷/린트/타입/테스트 명령 정의
- Linux · Windows CI, 보호된 main 워크플로, 물리·아키텍처 계약 문서

Phase 1 — 물리 문제 합의 ☒ 진행 중 (현재 단계)

- ☒ 프로토타입 전역 좌표계·크랭크각 규약 정의
- ☒ 실린더·로터·원형 컷아웃·베인 형상 문서화 (상단 위치에서 로터 중심-베인 팁 거리 9 mm 포함)
- ☐ 실린더, 상단 베인, 컷아웃, 로터를 압축기 CAD와 대조 확인
- ☐ 보이지 않는 베인-팁 기준 관계와 틈새를 CAD와 대조 확인
- ☐ 자유도(DOF), 구속조건, 필요한 출력 결정
- ☐ 기호·SI 단위·출처·범위가 있는 파라미터 표 작성
- ☐ 모든 힘을 목록화하고 처방(prescribed) vs. 계산(solved) 구분
- ☐ 독립 검증 가능한 기준(reference) 케이스 최소 1개 선정

종료 조건: 두 협업자가 코드를 보지 않고도 간단한 공유 케이스에서 각 힘의 부호와 크기 자릿수(order of magnitude)를 계산할 수 있을 것.

Phase 2 — 기준(reference) 동역학 솔버

1. 검증된 configuration과 평면 상태(state) 자료구조 구현
2. 명시적 zero-force 케이스를 포함한 모듈형 힘 조립(force assembly)
3. 진단(diagnostics)이 보이는 SciPy 기준 적분기 — `solve_ivp`, 명시적 방법 + 상대/절대 공차 보고, 출력 샘플링은 내부 스텝과 독립
4. 무력(zero-force) → 상수력 → 선형 진동자 해석해 테스트 통과
5. 결과 객체 + 최소한의 버전 관리되는 케이스/결과 포맷
6. 공차/스텝 정밀화(refinement) 수렴 시연

종료 조건: 압축기 특화 숨은 가정 없이 해석 케이스를 통과하고, 완전히 추적 가능한 결과를 산출할 것.

Phase 3 — 압축기 모델

- ☒ 처방(prescribed) 로터리 압축기 테스트 운동학 구현·검증
- ☒ 물리 속도/표시 속도가 분리된 인터랙티브 테스트 GUI
- ☒ 시계방향·상단 0도 운동, 연속 파라메트릭 로터·실린더-베인 윤곽
- ☒ 로터 립 양쪽 필렛, 원형 컷아웃에서 끝나는 개구부, 투명한 가변 길이 베인 오버레이
- ☒ 실린더 중심, 이동 로터 중심, 점선 편심 로터 중심 궤적 표시
- ☐ 합의된 하중/힘 모델을 **한 번에 하나씩** 추가
- ☐ 접촉·마찰은 수학적 처리 방식이 선정된 **후에만** 추가
- ☐ 경쟁 구현은 동일한 케이스에서 비교
- ☐ 선택된 변경만 integration 브랜치에서 조립

종료 조건: 결합 모델이 합의된 압축기 기준 케이스 허용 한계를 충족할 것.

Phase 4 — 견고성과 릴리스

1. 파라미터 스왑과 엣지 케이스 정의
2. 최적화 전에 대표 워크로드 프로파일링
3. 보존량(conservation) 및 실패 모드 회귀 테스트 추가
4. 첫 안정 입력 스키마와 공개 API 동결·문서화
5. 재현 가능한 릴리스 태그

검증 사다리 (PHYSICS.md §7 — 반드시 이 순서대로 진행)

① 무력 → ② 상수력 → ③ 선형 진동자 → ④ 좌표계/부호 케이스 →
⑤ 스텝/공차 정밀화 → ⑥ 에너지/일 균형 → ⑦ 압축기 기준 케이스

회귀 공차는 측정된 수치 오차를 반영해야 하며, 현재 출력을 통과시키기 위해 선택해서는 안 된다.

제2부 — 코드 작성 규약

2.1 Python 규약 (AGENTS.md)

- Python 3.11+, Windows·Linux 모두 지원 — 경로는 `pathlib.Path`, 플랫폼 특화 셸 가정 금지
- 공개·내부 인터페이스 모두 **타입 힌트** 필수
- **작은 순수 함수 + 불변(immutable) config/state 레코드** 선호 — 경계에서는 `dataclass` + NumPy 배열, 수치 코어에 비구조화 `dict` 전달 금지
- 네이밍: 모듈/함수/변수 `snake_case`, 클래스 `PascalCase`, 진짜 상수만 `UPPER_SNAKE_CASE`
- 주석은 **물리적·수치적 이유**를 설명 — 문법 재서술 금지
- 표준 라이브러리나 기존 의존성으로 해결되면 새 의존성 추가 금지
- 공개 API는 좁게 — `mochi.__init__` 에서 의도된 사용자 대상 이름만 재수출(re-export)
- NumPy 벡터 연산은 명료성이 향상될 때만 — 조기 최적화로 단순한 수식을 흐리지 말 것

2.2 물리·수치 규약 (AGENTS.md, PHYSICS.md)

- 내부는 **SI 단위 + 라디안** — 단위 변환은 입출력 경계에서만, 모든 변환 문서화. 각도 입력마다 0점 방향과 양의 회전 방향 명시
- 수식 구현 **전에** 좌표계, 양의 방향, 각도 원점, 힘-작용-물체(force-on-body) 규약을 먼저 정의
- 기본 `float64` 배열 — 다른 정밀도는 측정된 근거 + 회귀 테스트 필요
- 상태 변수는 명시적으로 — 모듈 레벨 가변 상태와 숨은 단위 변환 금지
- **모델 방정식 ↔ 시간 적분 ↔ 설정 ↔ 파일 I/O 분리**
- 힘은 독립적으로 테스트 가능한 기여분 `F_i(t, q, v, case data)` 로 표현하고, 어느 물체가 가하고 어느 물체가 받는지, 좌표계, 단위, 유효범위, 출처, 독립 테스트를 문서화
- 부재한 모델은 0을 기여하고 케이스 메타데이터에 보고 — 조용히 근사 금지
- 잘못된 형상·비물리적 입력은 solve 전에 유용한 에러로 조기 차단
- NaN, 무한값, 허용 한계 초과 접촉 관입, 솔버 비수렴은 절대 조용히 통과 금지 — 수렴 공차와 실패를 가시화
- 새 모델항마다 방정식·가정·파라미터 단위·출처(유도)·유효범위를 **같은 PR에서 PHYSICS.md에 추가**
- 수치 변경은 해석해, 제작해(manufactured solution), 신뢰 기준 데이터, 또는 문서화된 정밀화 연구로 검증

2.3 아키텍처 규약 (ARCHITECTURE.md)

```
configuration → model/state → physics/forces → numerics/integrators
                                     ↳ simulation orchestration → I/O
CLI → configuration과 orchestration만
```

- 의존성은 안쪽(state/model 인터페이스)으로만 향함
- 물리 코드는 CLI·출력 계층을 import하지 않음; 파일을 읽거나 명령행 인자를 파싱하지 않음
- 적분기는 미분/잔차(derivative/residual) 인터페이스로만 동작 — 압축기 특화 세부 알지 못함

- 빈 모듈을 미리 만들지 말 것 — 동작이 구현·테스트된 뒤에만 모듈 추가
- 테스트는 패키지 경계를 미리링
- 설정은 적분 전에 한 번만 검증 — 시간 루프에서 문자열 파싱·단위 추론 반복 금지
- 생성 결과물은 `results/` (커밋 금지), 기준 데이터는 출처·단위와 함께 `tests/data/` 에만 (작고 의도적이며 추적 가능해야 함)
- 성능: 명료한 벡터화 NumPy/SciPy로 시작, 측정 후 최적화. 빠른 알고리즘 추가 시 올바른 기준 구현 유지

2.4 Git · 협업 규약 (CONTRIBUTING.md, AGENTS.md)

브랜치 접두사	용도	예시
<code>feature/<owner>/<scope></code>	합의된 방향의 제품 작업	<code>feature/sh/kinematics</code>
<code>experiment/<owner>/<scope></code>	경쟁·불확실한 접근	<code>experiment/yk/contact-penalty</code>
<code>fix/<owner>/<scope></code>	결함 수정	<code>fix/sh/angle-sign</code>
<code>integration/<scope></code>	선택된 변경 조립 (단명)	<code>integration/contact-model</code>
<code>docs/<owner>/<scope></code>	문서 전용 작업	<code>docs/yk/force-conventions</code>

- **main은 보호됨**: 직접 개발 금지. PR + 상대 협업자 승인 + Linux/Windows CI + 대화 해결 필수. 항상 설치·테스트 가능해야 함
- 브랜치당 하나의 일관된 작업, 항상 최신 `main` 에서 시작 (`git switch main && git pull --ff-only && git switch -c ...`)
- 커밋: 명령형 메시지, cherry-pick 가능한 크기의 논리적 단위. 힘 모델 + 무관한 포매팅 + 생성 결과물 한 커밋에 섞지 말 것. 커밋 전 `git diff --staged` 확인
- 병렬 구현 선택: 두 후보 브랜치를 통째로 머지하지 말고, integration 브랜치에 **선택된 커밋만 cherry-pick**. 부분 채택 시 수동 포팅 후 출처 브랜치·커밋 해시를 PR에 기록
- 물리 충돌 해결: "ours/theirs" 금지 — 부호·단위·좌표계·가정 확인, 의도한 방정식 기록, 대안을 구별하는 테스트 추가, PHYSICS.md 기준으로 해결
- rebase는 개인 브랜치만 — 공유 히스토리 재작성·강제 push 금지
- 가상환경, 캐시, 커버리지, 로컬 에디터 설정, `results/`, 대용량 아티팩트 커밋 금지

2.5 로컬 검사 명령 (PR 전 전부 통과 필수)

```
ruff format --check .
ruff check .
mypy src/mochi
pytest
```

개발 중에는 관련된 가장 작은 테스트를 실행하고, 작업 완료 선언 전에 전체 검사를 실행한다. 변경을 통과 시키기 위해 공차를 약화하거나 회귀 케이스를 삭제하지 말 것.

2.6 완료 기준 (Definition of Done)

1. 의도된 동작이 구현됨
2. 관련 테스트 통과 + 전체 로컬 검사 세트 통과
3. 물리 문서(PHYSICS.md)와 사용자 문서가 코드와 일치
4. 실패 모드가 명시적
5. diff에 무관한 편집이나 생성 파일이 없음
6. 실질적 변경은 `CHANGES.md`에 의미 있는 항목 추가

다음 실작업 요약: Phase 1 미완 항목(CAD 대조 확인, 자유도/구속조건 결정, 파라미터 표, 힘 목록화, 기준 케이스 선정)이 우선이며, 코드 구현으로는 Phase 2의 config/state 자료구조 → zero-force 힘 조립 → SciPy 기준 적분기 순서가 첫 대상이다.